

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Трекинг множественных объектов»

Специальность ИИ-26(2)

Выполнил
А. В. Семёнов,
студент группы ИИ-26

Проверила
Андренко К.В.

Брест 2026

Цель работы: исследовать применение алгоритмов трекинга на базе обученной сети-детектора объектов.

Задание 1. Детектор и исходные видеоматериалы

Вариант 16: YOLO26s, класс cat. Для отслеживания использованы лучшие веса best.pt, полученные в лабораторной работе №3 на Cats v3. Детектор повторно не обучается. Требуется реализовать многообъектный трекинг, применить ByteTrack и BoT-SORT и исследовать влияние их параметров.

В лабораторной работе №3 модель обучалась 30 эпох с входом 416×416; лучшие веса выбраны на эпохе 13 по validation mAP50–95. На test получены mAP50 = 0,9656 и mAP50–95 = 0,6602. Эти значения характеризуют обнаружение на фотографиях Cats, а не сохранение идентификаторов на видео.

Использованы два открытых видеоролика с Wikimedia Commons. Для А выбран фрагмент 0–15 с, для В — 63–78 с, где видны несколько животных. Все конфигурации обрабатывают одни и те же кадры выбранного фрагмента в исходном порядке, без пропуска и без изменения скорости воспроизведения.

Обозначение	Видеоролик и автор	Фрагмент
А	Three orange kittens following their mother; Jeromi Mikhael [2]	0–15 с; 450 кадров; 30 кадр/с
В	Cat and kittens; Catfan [3]	63–78 с; 450 кадров; 29,970 кадр/с

В ролике А взрослая кошка и котята перемещаются возле ворот; камера движется, животные перекрывают друг друга и частично выходят за границы кадра. В ролике В наблюдается другая сцена с кошкой и котятами. Одновременное присутствие нескольких объектов позволяет проверить присвоение ID и сохранение траекторий.

Для А загружена версия VP9 480p (854×480), для В — исходный WebM VP8 (854×480). Перед детектором ширина ограничивается 854 пикселями с сохранением пропорций, затем Ultralytics выполняет масштабирование до входного размера $imgsz = 416$. Координаты CSV относятся к кадру выходного видео. Из-за округления числа кадров второй фрагмент длится 15,015 с.

Подготовка программы

Вся реализация находится в lab4.py. Программа находит веса ЛР 3, загружает видеоматериалы, создаёт отдельные YAML-файлы и запускает трекинг. Исходный датасет и Roboflow для четвёртой работы не требуются. Для воспроизводимости сохраняются SHA-256 весов и видео, версии библиотек и аргументы запуска.

Эксперимент выполнен с Python 3.12.14, PyTorch 2.7.1+cu126, Ultralytics 8.4.14 на NVIDIA GeForce RTX 4060. Использованы одни и те же веса для всех запусков. ReID отключён, дополнительные модели распознавания внешности не загружаются.

Задание 2. Реализация ByteTrack и BoT-SORT

Трекинг связывает обнаруженные рамки между соседними кадрами и присваивает объекту идентификатор. Фильтр Калмана прогнозирует состояние рамки; сопоставление связывает прогноз с текущими детекциями. Если объект временно не найден, его состояние сохраняется в списке потерянных треков.

ByteTrack выполняет сопоставление в два этапа. Сначала используются детекции с confidence не ниже `track_high_thresh`. Затем оставшиеся активные треки сопоставляются с менее уверенными рамками между `track_low_thresh` и `track_high_thresh`. Это позволяет продолжить трек при временном снижении уверенности детектора [1, 4].

BoT-SORT дополняет эту схему другим представлением состояния рамки и компенсацией движения камеры. В базовом опыте используется `sparseOptFlow`: глобальное преобразование оценивается по оптическому потоку. ReID является отдельной возможностью; при `with_reid = False` сходство внешнего вида не используется [1, 5].

```
model = YOLO(str(weights))
result = model.track(frame, persist=True,
                    tracker=str(config_path), imgsz=416,
                    conf=0.05, device=device, verbose=False)[0]
```

`persist=True` сохраняет состояние трекера между последовательными кадрами одного ролика. Для каждой пары «видео — конфигурация» создаётся новый экземпляр YOLO, поэтому ID, потерянные треки и параметры Калмана не переносятся из предыдущего опыта.

Из `result.bboxes` извлекаются ID, координаты, confidence и класс. Обработываются только рамки с присвоенным ID: обычная детекция без подтверждённого трека не считается активным треком. Центры последних 60 наблюдений соединяются линией; цвет определяется ID.

Сохраняемые результаты

Каждый опыт создаёт `tracked.mp4` без звука, `tracks.csv` с покадровыми координатами, `summary.json`, списки кадров для каждого ID и снимки через каждые три секунды. Частота кадров выходного MP4 совпадает с частотой источника. `comparison.csv` и `comparison.png` объединяют показатели всех конфигураций.

```
frame,time_s,id,class,confidence,x1,y1,x2,y2,
source_frame,source_time_s
```

Номер кадра фрагмента начинается с нуля, время равно `frame / fps`. Дополнительные поля `source_frame` и `source_time_s` указывают положение в исходном ролике. Один и тот же ID имеет смысл только внутри конкретного запуска. ID = 1 в двух видео не означает одно животное. Проверяются открытие видео, наличие весов и допустимость параметров.

Задание 3. Исследование параметров

Параметр	База	Назначение
track_high_thresh	0,25	Порог первого этапа сопоставления
track_low_thresh	0,10	Нижний порог слабых детекций
new_track_thresh	0,25	Порог создания нового трека
track_buffer	30	Число шагов сохранения потерянного трека
match_thresh	0,80	Максимальная стоимость первого сопоставления
fuse_score	True	Учёт confidence в стоимости сопоставления
gmc_method	sparseOptFlow	Компенсация камеры только для BoT-SORT
with_reid	False	Сопоставление по внешнему виду отключено

Для каждого алгоритма выполнены пять опытов. Варианты buffer5 и buffer90 изменяют только track_buffer; match50 — только match_thresh до 0,50; high60 — только track_high_thresh до 0,60. Дополнительный опыт botsort_no_gmc отключает компенсацию камеры. Всего: 11 конфигураций на двух роликах, 22 запуска.

Порог детектора $\text{conf} = 0,05$ выбран ниже track_low_thresh = 0,10. Иначе слабые рамки были бы удалены ещё до второго этапа трекера. Повышение track_high_thresh при неизменном new_track_thresh также затрудняет запуск новых треков: новые кандидаты формируются из группы сильных детекций.

match_thresh ограничивает стоимость, а не непосредственно IoU. При fuse_score=True confidence участвует в стоимости. Поэтому уменьшение match_thresh с 0,80 до 0,50 делает первый этап более строгим. Внутренние пороги второго этапа и подтверждения новых треков остаются такими, как в Ultralytics 8.4.14.

В этой версии model.track создаёт трекер с frame_rate = 30. При последовательной обработке кадров значение track_buffer соответствует числу шагов обновления. На ролике с 30 кадр/с 30 шагов составляют примерно секунду; для другой частоты длительность равна track_buffer / fps. Потерянная рамка не рисуется как обнаруженная до нового сопоставления.

Методика оценки

По CSV рассчитаны: число уникальных ID; среднее число активных треков на кадр; число кадров без активных треков; число кадров с двумя и более треками; число повторных появлений того же ID после пропуска хотя бы одного кадра. Последний показатель обозначается G и не является числом смен ID.

У роликов нет покадровой эталонной разметки траекторий. Поэтому MOTA, IDF1 и число ID switches не вычисляются. Количественные показатели используются вместе с просмотром кадров. Меньше ID не обязательно лучше: результат может объясняться пропусками животных. Выводы относятся к выбранным фрагментам, а не ко всем видео с кошками.

Количественные результаты

А — кошка с рыжими котятами

Конфигурация	ID	Среднее	Пустые	≥2	G
bytetrack_base	6	1,03	20	35	17
bytetrack_buffer5	8	1,02	20	31	13
bytetrack_buffer90	5	1,03	20	35	18
bytetrack_match50	20	0,85	78	11	30
bytetrack_high60	2	0,93	33	3	10
botsort_base	7	1,03	20	32	13
botsort_buffer5	9	1,02	20	30	10
botsort_buffer90	6	1,03	20	32	14
botsort_match50	22	0,85	76	10	22
botsort_high60	2	0,91	41	0	6
botsort_no_gmc	7	1,03	17	31	13

В — кошка и котята

Конфигурация	ID	Среднее	Пустые	≥2	G
bytetrack_base	10	0,98	47	39	27
bytetrack_buffer5	13	0,97	48	32	20
bytetrack_buffer90	10	0,98	47	39	27
bytetrack_match50	34	0,69	162	21	27
bytetrack_high60	5	0,90	57	11	10
botsort_base	14	0,98	46	36	16
botsort_buffer5	17	0,96	47	30	10
botsort_buffer90	13	0,98	46	37	17
botsort_match50	38	0,70	155	17	25
botsort_high60	5	0,76	118	11	8
botsort_no_gmc	13	0,98	47	36	17

ID — число выданных идентификаторов; «Среднее» — активные треки на кадр; «Пустые» — кадры без ID; «≥2» — кадры с несколькими ID; G — возвраты того же ID после пропуска. Это описательные показатели результата трекера, а не эталонные метрики точности.

Сравнение конфигураций

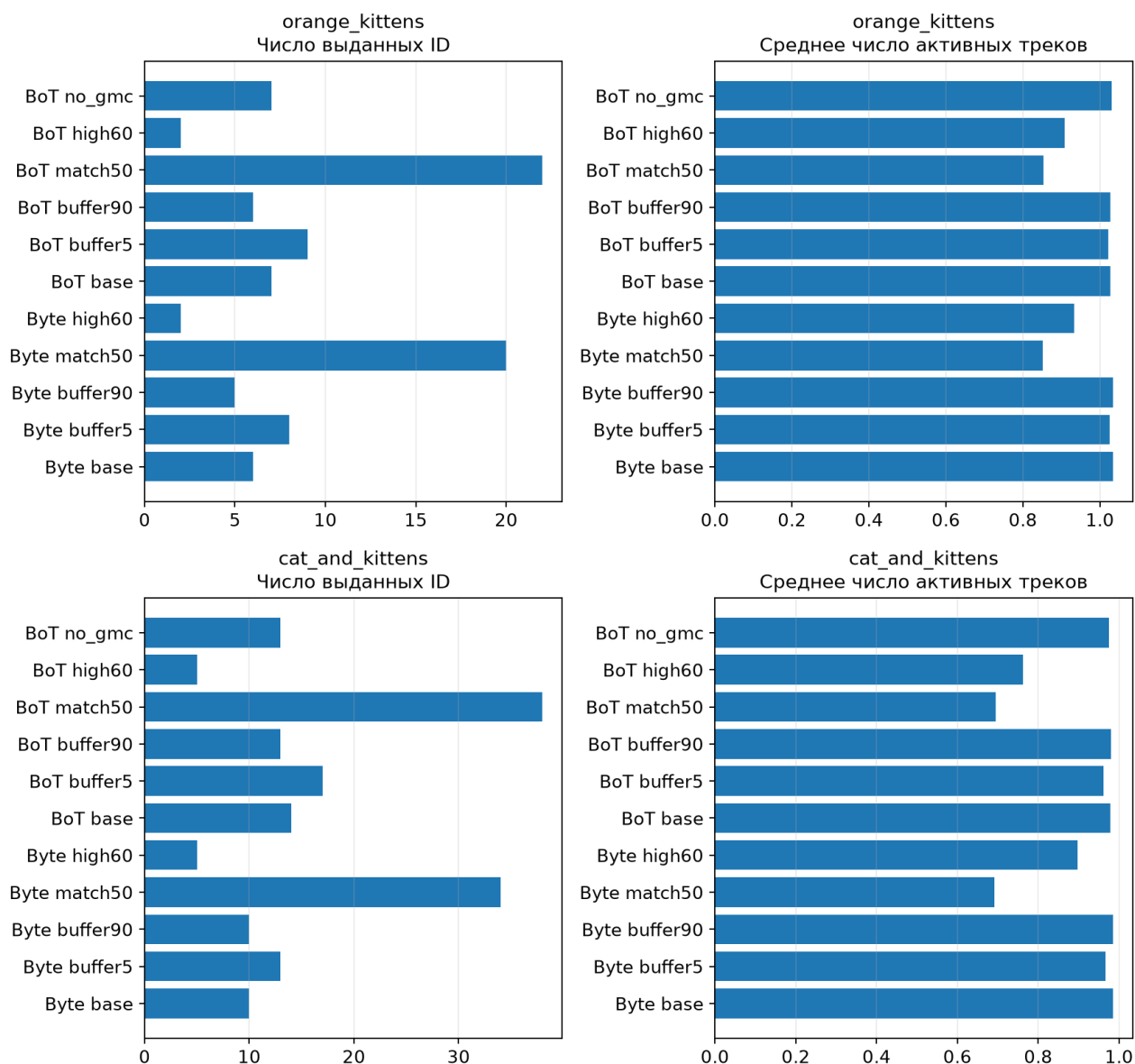


Рисунок 1 — Число идентификаторов и активных треков

На ролике A ByteTrack в базовом режиме выдал 6 ID, BoT-SORT — 7. Среднее число активных треков составило 1,03 и 1,03. При `match_thresh = 0,50` ByteTrack выдал 20 ID; кадров без активных треков стало 78 вместо 20.

На ролике B ByteTrack в базовом режиме выдал 10 ID, BoT-SORT — 14. Среднее число активных треков составило 0,98 и 0,98. При `match_thresh = 0,50` ByteTrack выдал 34 ID; кадров без активных треков стало 162 вместо 47.

Слишком строгий порог может отклонять корректное соответствие после движения камеры или изменения рамки. Возникают пропуски и новые идентификаторы. Увеличение буфера оставляет больше времени на восстановление, но само по себе не исправляет ошибочные рамки и не гарантирует правильную идентичность объекта.

Примеры трекинга на ролике А



Рисунок 2 — ByteTrack, первый кадр с двумя ID, $t = 0,67$ с



Рисунок 3 — ByteTrack, фрагмент А на девятой секунде

На кадрах показаны только подтверждённые ID, а не все слабые ответы детектора. Подпись содержит класс, идентификатор и confidence. Линии отображают историю центров рамок. Котята меньше взрослой кошки, частично перекрыты и обрезаны границами изображения, поэтому их обнаружение менее устойчиво.

Рисунок 2 — первый кадр, на котором выданы два ID: взрослой кошке и одному котёнку. Рамка взрослого животного слишком велика. На рисунке 3 кошка сохраняет ID = 1, но видимые котята не имеют отдельных треков. Настройка буфера не исправляет пропуски и неточную локализацию детектора.

Примеры трекинга на ролике В



Рисунок 4 — BoT-SORT, около 66-й секунды исходного видео В

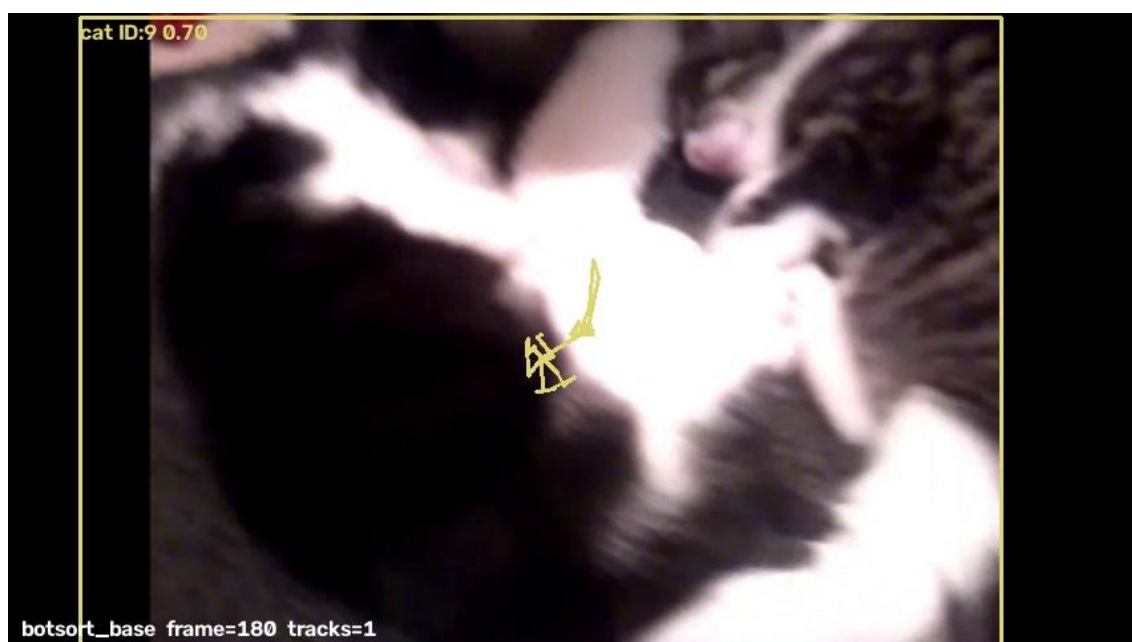


Рисунок 5 — BoT-SORT, около 69-й секунды исходного видео В

Показаны кадры через три и шесть секунд от начала фрагмента. Соседние животные ошибочно объединены общей рамкой с ID = 9. Сохранение этого ID не означает правильное раздельное сопровождение кошек. Во фрагменте меняются ракурс и масштаб; монтажные переходы ближе к концу также нарушают непрерывность треков.

Кадры ролика В воспроизведены из работы Catfan «Cat and kittens», CC BY-SA 3.0 [3]. Изменения: выделен фрагмент, нанесены рамки, ID и траектории. Эти производные изображения и размеченное видео распространяются на условиях CC BY-SA 3.0.

Анализ результатов и ограничения

Ролик А: уменьшение буфера ByteTrack до 5 изменило число ID с 6 до 8, увеличение до 90 — до 5. При пороге сильных детекций 0,60 число ID равно 2, а кадров с несколькими треками — 3 вместо 35. Последний результат нужно оценивать вместе с видимостью котят, а не считать уменьшение числа ID самостоятельным улучшением.

BoT-SORT на ролике А: при отключении GMC число ID осталось равным 7, число пустых кадров изменилось с 20 до 17. Эффект компенсации зависит от сцены: оптический поток по движущимся животным и перекрытия могут ухудшать оценку движения камеры.

Ролик В: уменьшение буфера ByteTrack до 5 изменило число ID с 10 до 13, увеличение до 90 — до 10. При пороге сильных детекций 0,60 число ID равно 5, а кадров с несколькими треками — 11 вместо 39. Последний результат нужно оценивать вместе с видимостью котят, а не считать уменьшение числа ID самостоятельным улучшением.

BoT-SORT на ролике В: при отключении GMC число ID изменилось с 14 до 13, число пустых кадров изменилось с 46 до 47. Эффект компенсации зависит от сцены: оптический поток по движущимся животным и перекрытия могут ухудшать оценку движения камеры.

Скорость обработки

Видео	ByteTrack, кадр/с	BoT-SORT, кадр/с
А	95,0	29,8
В	92,4	51,3

Измерено время вызова `model.track` после первого кадра, который включает инициализацию. Чтение, рисование и запись видео в эту скорость не входят. Каждый опыт выполнен один раз; измерение зависит от оборудования и текущей нагрузки и не является отдельным тестом производительности.

Практический вывод

Базовые параметры служат отправной точкой. Слишком строгий `match_thresh` увеличивает риск разрыва траектории; короткий буфер уменьшает возможность восстановления после пропуска. При повышении `track_high_thresh` часть животных перестаёт получать ID. По выбранным роликам нельзя установить универсальное превосходство одного алгоритма.

Для полноценной оценки качества нужны покадровые рамки с постоянными эталонными ID, отдельные видео для настройки и проверки, а также подсчёт IDF1 или НОТА. Для текущего детектора дополнительно полезны обучающие примеры маленьких и перекрытых котят и сцен с несколькими кошками.

Спецификация ввода и вывода

Весь код находится в lab4.py. Требуются Python 3.10 или новее, библиотеки и best.pt из ЛР 3. При переносе на другой компьютер нужны и веса: они не встроены в Python-файл.

```
python -m pip install ultralytics==8.4.14 lap==0.5.12 matplotlib
python lab4.py
```

Без аргументов веса ищутся рядом, затем в results_lab3/*/train/weights/best.pt; выбирается последний запуск по имени папки. Программа загружает два видео и выполняет 22 опыта. Имеющиеся видео используются повторно.

```
python lab4.py --weights "best.pt"
python lab4.py --source "cats.mp4" --seconds 0
python lab4.py --configs botsort_base --show
python lab4.py --configs bytetrack_base --set track_buffer=60
```

Параметр --start задаёт начало фрагмента, --seconds — длительность; 0 означает обработку до конца. Для демонстрации по умолчанию А начинается с 0 с, В — с 63 с; для своих файлов начало равно 0. --source принимает файлы или прямые HTTP(S)-ссылки на видео. --device cpi включает процессор, --device 0 — первую видеокарту. При --show клавиша Q завершает текущий опыт.

Новые результаты сохраняются в отдельной папке results_lab4 с датой и временем. В ней находятся manifest.json, configs/*.yaml, comparison.csv, comparison.png, summary.json и подпапки с видео, кадрами и tracks.csv. Веса и исходные видео не изменяются. Кадры с отсутствующими ID учитываются в статистике как пустые.

Вывод

Реализовал отслеживание нескольких объектов класса cat на основе YOLO26s, обученного в ЛР 3. Применил ByteTrack и BoT-SORT, создал 11 конфигураций и проверил их на двух фрагментах интернет-видео. Исследовал влияние буфера потерянных треков, порогов и компенсации движения камеры. Полученные результаты показывают зависимость устойчивости ID как от настроек сопоставления, так и от качества обнаружения животных.